

Report: Data Structures Empirical Benchmark & Algorithmic Analysis

Course: Design and Analysis of Algorithms (Assignment 2)
Environment: Java OpenJDK 17+, `System.nanoTime()`, `RUNS = 5` (Warmup pass excluded, Median reported)

1. Theoretical Complexity Comparison

Data Structure	Operation	Best Case	Average Case	Worst Case	Space Complexity
DynamicArray	Random Access <code>get(i)</code>	$O(1)$	$O(1)$	$O(1)$	$O(N)$
	Search <code>contains(x)</code>	$O(1)$	$O(N)$	$O(N)$	$O(N)$
	Insert/Remove (Head)	$O(N)$	$O(N)$	$O(N)$	$O(N)$
	Insert/Remove (Middle)	$O(N)$	$O(N)$	$O(N)$	$O(N)$
MyLinkedList	Random Access <code>get(i)</code>	$O(1)$	$O(N)$	$O(N)$	$O(N)$
	Search <code>contains(x)</code>	$O(1)$	$O(N)$	$O(N)$	$O(N)$
	Insert/Remove (Head)	$O(1)$	$O(1)$	$O(1)$	$O(N)$
	Insert/Remove (Middle)	$O(1)$	$O(N)$	$O(N)$	$O(N)$
MinHeap	Insert <code>insert(x)</code>	$O(1)$	$O(\log N)$	$O(\log N)$	$O(N)$
	Extract Min <code>extractMin()</code>	$O(1)$	$O(\log N)$	$O(\log N)$	$O(N)$

2. Workload 1: Random Access (`w1_random_access`)

Execution of 10,000 random read operations (`get(index)`).

Workload	Variant	Structure	N	Time (ms)	Steps	Moves	Comparisons
<code>w1_random_access</code>	N/A	DynamicArray	100	0.092	10,000	0	0
<code>w1_random_access</code>	N/A	DynamicArray	1000	0.105	10,000	0	0
<code>w1_random_access</code>	N/A	DynamicArray	10000	0.210	10,000	0	0
<code>w1_random_access</code>	N/A	DynamicArray	100000	0.470	10,000	0	0
<code>w1_random_access</code>	N/A	MyLinkedList	100	0.350	502,100	0	0
<code>w1_random_access</code>	N/A	MyLinkedList	1000	3.120	5,012,300	0	0
<code>w1_random_access</code>	N/A	MyLinkedList	10000	48.500	50,089,000	0	0
<code>w1_random_access</code>	N/A	MyLinkedList	100000	729.970	505,412,000	0	0

Empirical Insights & Analysis for W1:

- **DynamicArray:** Access time remains constant (**0.092–0.470 ms**) and the step count is strictly fixed at **10,000** across all input sizes N . This confirms the theoretical $O(1)$ random access complexity via direct memory address calculation: $\text{Address} = \text{Base} + i \times 4 \text{ bytes}$.
- **MyLinkedList:** The step count grows proportionally to $\approx 10,000 \times \frac{N}{2}$. At $N = 100,000$, the number of traversed nodes exceeds **505 million**, driving execution time up to **729.970 ms** (~1,550 times slower than **DynamicArray**), demonstrating true $O(N)$ linear traversal cost.

3. Workload 2: Linear Search (w2_search)

Execution of 1,000 search operations for target values (`contains(value)`).

Workload	Variant	Structure	N	Time (ms)	Steps	Moves	Comparisons
w2_search	N/A	DynamicArray	100	0.045	51,200	0	51,200
w2_search	N/A	DynamicArray	1000	0.310	508,400	0	508,400
w2_search	N/A	DynamicArray	10000	2.850	5,123,000	0	5,123,000
w2_search	N/A	DynamicArray	100000	22.990	74,565,647	0	74,565,647
w2_search	N/A	MyLinkedList	100	0.090	51,200	0	51,200
w2_search	N/A	MyLinkedList	1000	0.820	508,400	0	508,400
w2_search	N/A	MyLinkedList	10000	11.400	5,123,000	0	5,123,000
w2_search	N/A	MyLinkedList	100000	131.087	74,565,647	0	74,565,647

Empirical Insights & CPU Cache Analysis for W2:

- **Identical Operation Counts:** The number of comparisons is identical across both structures for every N (e.g., **74,565,647** at $N = 100,000$).
- **Spatial Locality & Cache Hits:** At $N = 100,000$, **DynamicArray** executes in **22.990 ms**, whereas **MyLinkedList** takes **131.087 ms** (~5.7x faster). Contiguous memory layout allows **DynamicArray** to prefetch full CPU cache lines (64 bytes = 16 integers). Dispersed Heap objects in **MyLinkedList** incur constant L3 cache misses during pointer chasing.

4. Workload 3: Insert & Remove (w3_insert_remove)

Execution of 1, 000 insertion and deletion cycles.

Head Variant (index = 0)

Workload	Variant	Structure	N	Time (ms)	Steps	Moves	Comparisons
w3_insert_remove	head	DynamicArray	100	0.080	0	200,000	0
w3_insert_remove	head	DynamicArray	1000	0.350	0	2,000,000	0
w3_insert_remove	head	DynamicArray	10000	1.850	0	20,000,000	0
w3_insert_remove	head	DynamicArray	100000	12.291	0	201,000,000	0
w3_insert_remove	head	MyLinkedList	100	0.005	0	2,000	0
w3_insert_remove	head	MyLinkedList	1000	0.006	0	2,000	0
w3_insert_remove	head	MyLinkedList	10000	0.008	0	2,000	0
w3_insert_remove	head	MyLinkedList	100000	0.011	0	2,000	0

Middle Variant (index = N/2)

Workload	Variant	Structure	N	Time (ms)	Steps	Moves	Comparisons
w3_insert_remove	middle	DynamicArray	100	0.050	0	100,000	0
w3_insert_remove	middle	DynamicArray	1000	0.210	0	1,000,000	0
w3_insert_remove	middle	DynamicArray	10000	1.100	0	10,000,000	0
w3_insert_remove	middle	DynamicArray	100000	5.875	0	100,500,000	0
w3_insert_remove	middle	MyLinkedList	100	0.120	100,000	2,000	0
w3_insert_remove	middle	MyLinkedList	1000	1.150	1,000,000	2,000	0
w3_insert_remove	middle	MyLinkedList	10000	15.400	10,000,000	2,000	0
w3_insert_remove	middle	MyLinkedList	100000	190.156	100,000,000	2,000	0

Empirical Insights & Analysis for W3:

- **Head Operations:** MyLinkedList achieves true $O(1)$ performance (**0.011 ms** at $N = 100,000$) with 0 traversal steps. DynamicArray at the same size must shift memory blocks (**201,000,000** element moves, taking 12.291 ms).
- **Middle Operations:** DynamicArray (**5.875 ms**) outperforms MyLinkedList (**190.156 ms**) by over **32 times**. High-performance block memory shifting via `System.arraycopy` is hardware-accelerated, whereas MyLinkedList suffers from **100,000,000** sequential node traversal steps.

5. Workload 4: Priority Queue / MinHeap (w4_priority_queue)

Sequential execution of N insertions (insert) followed by N minimum extractions (extractMin).

Workload	Variant	Structure	N	Time (ms)	Steps	Moves	Comparisons
W4_priority_queue	N/A	MinHeap	100	0.015	540	310	1,069
W4_priority_queue	N/A	MinHeap	1000	0.120	8,900	5,200	18,450
W4_priority_queue	N/A	MinHeap	10000	1.150	124,000	71,000	245,100
W4_priority_queue	N/A	MinHeap	100000	10.377	1,580,000	912,000	3,059,283

Empirical Insights for W4:

- **Logarithmic Scaling:** Scaling N by $1,000\times$ causes comparison counts to grow moderately from 1,069 to **3,059,283**, taking only **10.377 ms** for 100,000 elements. Empirical metrics align precisely with the theoretical $O(N \log N)$ bound.

6. Memory Overhead Analysis

- **DynamicArray:** Uses a dense primitive array `int[]`. Each element consumes exactly **4 bytes** (plus minor overhead from $1.5\times$ growth capacity buffer).
- **MyLinkedList:** Wraps each element in a `Node` object. Object header (12 bytes) + `next/prev` pointers (8–16 bytes) + value (4 bytes) + padding = **24–32 bytes per node**. Memory consumption is **6–8x higher** than `DynamicArray`.

7. Formal Loop Invariants

Invariant 1: `contains(x)` in `DynamicArray`

- **Statement:** At the start of iteration i , target element x is not present in `data[0 ... i-1]`.
- **Initialization:** For $i = 0$, subarray `data[0 ... -1]` is empty; the invariant holds vacuously.
- **Maintenance:** If `data[i] == x`, the loop terminates returning `true`. Otherwise $i \rightarrow i + 1$, maintaining $x \notin \text{data}[0 \dots i]$.
- **Termination:** Loop terminates when $i = \text{size}$. The invariant guarantees $x \notin \text{data}[0 \dots \text{size} - 1]$; method returns `false`.

Invariant 2: `siftDown(index)` in `MinHeap`

- **Statement:** Prior to each iteration, the min-heap property holds for all nodes in the tree, except possibly between the node at `index` and its children.
- **Initialization:** Swapping root with the last leaf element isolates potential order violation strictly to `index = 0`.
- **Maintenance:** If node at `index` violates heap order, it is swapped with its smaller child, pushing the potential violation down one level.
- **Termination:** Loop terminates when `index` has no children or is $\leq$ both children. Min-heap order is globally restored.

8. Practical Recommendation Matrix

Use Case Scenario	Recommended Structure	Justification
Frequent random access & search	DynamicArray	Constant $O(1)$ lookup time and superior CPU cache spatial locality.
Frequent insertions/deletions at Head	MyLinkedList	True $O(1)$ operations with zero memory shifting overhead.
Priority queues & task scheduling	MinHeap	Guaranteed $O(\log N)$ bounds for inserting and extracting minimum elements.